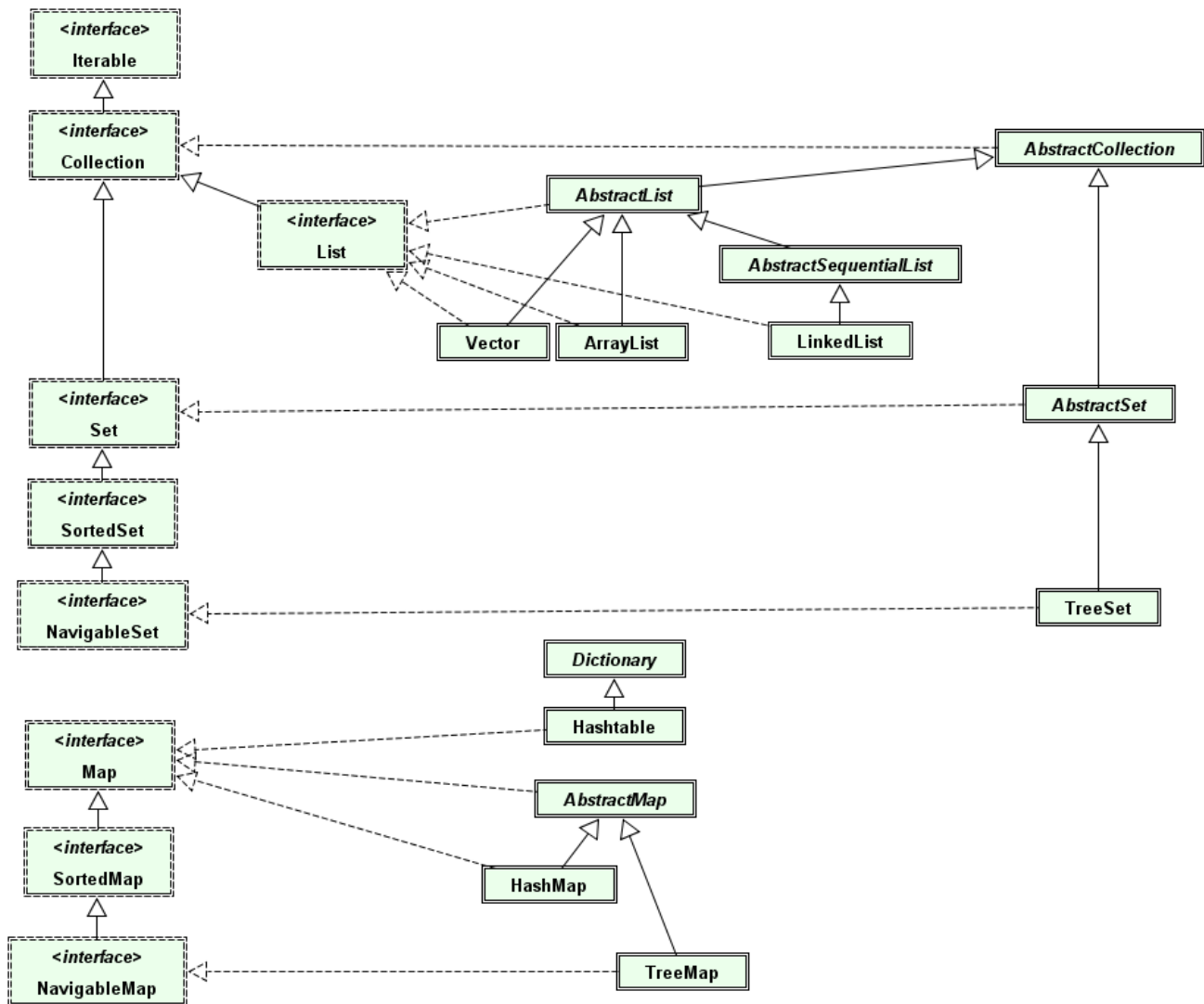


Collections & Maps

- Etablir l'arbre d'héritage des interfaces suivants : **Iterable**, **Collection**, **List**, **Map**, **Set**, **SortedMap**, **SortedSet**, **NavigableMap**, **NavigableSet**.
- Etablir l'arbre d'héritage des collections prédéfinies **AbstractCollection**, **AbstractList**, **AbstractSet**, **TreeSet**, **ArrayList**, **AbstractSequentialList**, **LinkedList**, **Vector**, **Dictionary**, **Hashtable**, **AbstractMap**, **HashMap**, **TreeMap**.



Pour les informations suivantes, voir :

<http://java.sun.com/developer/onlineTraining/collections/Collection.html>

le code source et JavaDoc de ces classes/interfaces.

- Qu'est-ce qu'une **Collection**? Quels sont ses caractéristiques ?

Une collection est un interface servant à définir des groupes d'objets qui sont accessibles comme une seule entité. Dans des collections, il peut exister des **doublons** (*duplicates*) des objets. **Collection** implique la définition de toute une série de méthodes pour ajouter et supprimer des éléments ainsi qu'un itérateur et une série de méthodes de test. Les éléments ne sont pas **ordonnés**.

d) Qu'est-ce qu'un **Set** ? Quelles sont ses caractéristiques (en comparant avec **Collection**) ?

Un **Set** est un ensemble dans le sens mathématique du terme : chaque élément n'existe qu'une seule fois et il ne peut y avoir qu'un seul élément **null**. **Il ne peut pas y avoir de doublons (*duplicate*)**. Les méthodes sont identiques à celles de **Collection**, mais elles ne permettent pas l'intégration de deux objets où `e1.equals(e2)`. Les éléments ne sont pas ordonnés.

Attention : Les objets ne doivent pas être changés (***immutable***), car les réactions de **Set** sont indéfinies si on change les attributs d'un de ses objets.

e) Qu'est-ce qu'une **List**? Quels sont ses caractéristiques ?

Une liste est une collection où les éléments sont **ordonnés** : chaque élément a une position (indice) dans la collection. Il peut y avoir des doublons et plusieurs éléments null.

Des méthodes supplémentaires facilitent la manipulation des objets par un index.

Un itérateur spécial (ListIterator) permet l'insertion, le remplacement et l'accès bidirectionnel aux objets.

f) Qu'est-ce qu'une **Map** ? Quelles sont ses caractéristiques et champs d'application ?

Une **Map** est un interface permettant de définir un ensemble (**Set**) qui contient des paires de données, où on **associe** dans chaque paire un objet d'un premier type à un objet d'un deuxième type. (P.ex. adresses IP et Noms/DNS, champs clés et enregistrements d'une BD, mots d'un dictionnaire, etc). **Map** est une définition à part qui n'est pas dérivée de **Collection** ou **Set**.

Map<K,V> associe des clés du type **K** (« *key* ») à des valeurs du type **V** (« *value* »).

Par définition, K ne doit pas avoir de doublons, pour cette raison, les méthodes traitant plusieurs clés à la fois fonctionnent avec des **Set**.

g) Qu'est-ce qu'une **HashMap** ? Quels sont ses champs d'application ?

HashMap est une implémentation de **Map**. Pour l'utiliser, il faut que la classe des clés (**K**) ait bien défini la méthode **hashCode()**.

HashMap est bien adapté à des applications où il faut insérer, effacer, localiser des éléments.

h) Quelles sont les caractéristiques des interfaces **SortedMap** et **SortedSet** ?

SortedSet et **SortedMap** gardent leurs éléments respectivement leurs clés dans un ordre défini. Pour que cela fonctionne, il faut que les classes implémentent **Comparable** ou fournissent un comparateur (***Comparator***).

i) Quelles sont les caractéristiques de **TreeMap** et **TreeSet** ?

TreeMap est une implémentation de **Map**.

TreeSet est une implémentation de **Set**.

Dans les deux cas, les données sont organisées de façon triée dans un arbre balancé. Pour pouvoir les utiliser, il faut que les clés (**K**) puissent être triées, c.-à-d. qu'ils implémentent **Comparable**.

Ils sont bien adaptés à des applications où il faut traverser les éléments en ordre (ou les localiser rapidement).

j) Quelles sont les différences entre **ArrayList** et **Vector** ?

Les fonctionnalités de **ArrayList** et **Vector** sont presque identiques, **Vector** est une implémentation 'historique' qui a été remplacée par **ArrayList**.

Différences :

- **ArrayList** n'est pas synchronisé, c.-à-d. **ArrayList** n'est pas 'thread-safe' : Une **ArrayList** n'est pas bloquée lors d'une opération de modification.
Vector est synchronisé, donc en principe *thread-safe*. En conséquence, les performances de **Vector** sont bien inférieures à celle de **ArrayList**.
- Les dimensions d'une **ArrayList** ou d'un **Vector** sont élargies automatiquement selon besoin. Les algorithmes d'élargissement de ces deux classes sont différents.

k) Quelles sont les différences entre **HashMap** et **Hashtable** ?

- **Hashtable** est synchronisé, **HashMap** ne l'est pas.
- **Hashtable** ne permet pas d'objet **null** ni dans les clés ni dans les valeurs. **HashMap** permet une clé (**K**) **null** et plusieurs valeurs (**V**) **null**.
- **HashMap** peut simplement être remplacé par sa sous-classe **LinkedHashMap** qui est ordonnée et qui fournit différents itérateurs pour parcourir les clés et les valeurs.

l) Que peut-on dire de **Vector** et **Hashtable** et de leur utilisation ?

Vector et **Hashtable** sont des implémentations 'historiques' qui ont été remplacées par **ArrayList** et **HashMap**. Il est fortement conseillé d'employer **ArrayList** et **HashMap**. NetBeans les marque comme '*deprecated*' (« déconseillé »).

Si on a besoin d'une liste synchronisée, il est conseillé d'employer **ArrayList** et de la synchroniser à l'aide de **Collections.synchronizedList(...)**.

m) Qu'est-ce que la **synchronisation** dans le contexte des collections ? Quand est-ce qu'elle joue un rôle ?

Si plusieurs threads accèdent à une structure et au moins l'une des deux modifie la structure, alors il faut qu'une structure soit synchronisée de l'extérieur. Sinon, on dit que l'accès n'est pas *thread-safe*.

n) Comment peut-on **synchroniser** une collection qui ne l'est pas dès le départ ?

En général, la synchronisation se fait par un objet extérieur qui contrôle les accès à la structure. Si un tel objet n'existe pas, alors on peut 'envelopper' la structure dès la création par les méthodes

Collections.synchronizedMap, **Collections.synchronizedSortedSet**,
Collections.synchronizedSortedMap, **Collections.synchronizedSet**,
Collections.synchronizedMap, **Collections.synchronizedList**,
Collections.synchronizedCollection.

Exemple :

Il faut alors accéder à la structure par l'itérateur et à l'intérieur d'un bloc synchronisé (synchronized). Par exemple :

```
Set s = Collections.synchronizedSet(new HashSet());
```

```
...
```

```
synchronized(s) {  
    Iterator i = s.iterator(); // Must be in the synchronized block  
    while (i.hasNext())  
        foo(i.next());  
}
```

Autre possibilité :

Utiliser les classes de **java.util.concurrent** :

P.ex **CopyOnWriteArrayList** est une ArrayList qui est tout à fait thread-safe sans nécessiter d'autres opérations.

Toutes les opérations mutatives (add, set, remove, etc) sont effectuées sur des copies de toute la liste.

Ceci peut être très coûteux, mais c'est pratique si on utilise la liste surtout en lecture.

Il existe un grand nombre de solutions dans le paquet **java.util.concurrent**, mais chaque solution a d'autres spécificités et il faut lire JavaDoc attentivement avant de pouvoir employer ces classes.